REPORTE TAREA 1
ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Vicente Vizcarra Rubio

29 de abril de 2025

03:56

Resumen

En esta tarea se implementaron y analizaron diversos algoritmos de ordenamiento y de multiplicación de matrices.

Se desarrollaron algoritmos como SelectionSort, MergeSort, QuickSort y std::sort para el ordenamiento de arreglos unidimensionales, y algoritmos de multiplicación de matrices como Naive y Strassen.

Se realizaron experimentos para medir el tiempo de ejecución y el consumo de memoria de cada algoritmo, utilizando conjuntos de datos de diferentes tamaños. Los resultados experimentales fueron graficados y analizados, observándose diferencias en el comportamiento de los algoritmos según el tamaño y tipo de entrada.

Índice

1. Introducción **2**

2. Implementaciones **3**

3. Experimentos **4**

4. Resultados experimentales **5**

5. Conclusiones **7**

A. Apéndice 1 **8**

1. Introducción

El análisis y diseño de algoritmos es un área fundamental en Ciencias de la Computación, debido a que permite evaluar la eficiencia de diferentes métodos para resolver problemas de carácter computacional. El estudio de la complejidad temporal y espacial de los algoritmos es esencial para poder elegir la solución más adecuada teniendo en cuenta los recursos disponibles y el tamaño de los datos de entrada.

Dentro de este contexto, los algoritmos de ordenamiento y de multiplicación de matrices son herramientas clásicas que han sido ampliamente estudiadas, debido a su aplicación en diversos dominios, como bases de datos, gráficos computacionales y aprendizaje automático. Si bien existen soluciones eficientes en teoría, en la práctica pueden variar según las condiciones de entrada y las características específicas de cada implementación.

El propósito de este informe es comparar experimentalmente diferentes algoritmos de ordenamiento (SelectionSort, MergeSort, QuickSort y std::sort) y de multiplicación de matrices (Naive y Strassen), evaluando su desempeño en términos del tiempo de ejecución y el uso de memoria. Este análisis busca identificar patrones de comportamiento y contrastar la eficiencia teórica con el rendimiento experimental, destacando fortalezas y debilidades de cada enfoque.

Este trabajo tiene como objetivo reforzar el entendimiento de los conceptos de complejidad de algoritmos, aplicándolos en un contexto práctico y experimental.

2. Implementaciones

https://github.com/VicenteVR/INF221-2025-1-TAREA-1.git

3. Experimentos

Para la realización de los experimentos, se utilizó un equipo con sistema operativo Windows 10 ejecutando Ubuntu 20.04.6 LTS a través del Subsistema de Windows para Linux (WSL). El procesador es un Intel Core i5-1135G7 a 2.40GHz 2.42 GHz con 8 GB de RAM. La compilación de los programas se realizó con el compilador `g++` versión 9.4.0, utilizando el estándar `-std=c++17` y la optimización `-O2`. Para la visualización de los resultados se empleó Python 3.8 junto con la biblioteca `matplotlib`.

Los experimentos se dividieron en dos áreas: algoritmos de ordenamiento y algoritmos de multiplicación de matrices. Para cada caso, se implementaron múltiples algoritmos para ser comparados bajo un mismo conjunto de datos.

Para el ordenamiento, se utilizaron:

- **SelectionSort**
- **MergeSort**
- **QuickSort**
- **std::sort** (función estándar de C++)

Para la multiplicación de matrices se implementaron:

- **Algoritmo Naive** (multiplicación clásica con tres ciclos anidados)
- **Algoritmo de Strassen**

Cada algoritmo fue evaluado respecto a:

- Tiempo de ejecución (en microsegundos o segundos según el contexto)
- Uso de memoria (en kilobytes)

Se desarrollaron scripts automatizados que ejecutaron los algoritmos sobre los archivos de entrada. Estos archivos fueron generados previamente por scripts en Python, simulando diferentes condiciones para evaluar el comportamiento de cada algoritmo en diversos casos. Para ordenamiento, se utilizaron arreglos de tamaños variados con orden aleatorio, ascendente y descendente. Para multiplicación de matrices se usaron matrices de tamaños de potencias de 2 con estructuras dispersas, diagonales y densas.

Cada experimento se repitió tres veces por configuración, almacenando los resultados individualmente para luego obtener un promedio. Los datos recolectados se procesaron para generar gráficos comparativos que facilitan el análisis visual del comportamiento de los distintos algoritmos.

3.1. Dataset (casos de prueba)

Para la evaluación de los algoritmos de ordenamiento y multiplicación de matrices se utilizaron los conjuntos de datos definidos en el enunciado de la tarea, generados mediante los programas en Python.

En el caso del ordenamiento de arreglos, se generaron arreglos de tamaños 10^1, 10^3, 10^5 y 10^7, con tres tipos de orden: ascendente, descendente y aleatorio. Para cada una de estas combinaciones de tamaño y tipo de orden, se generaron tres muestras distintas, permitiendo así observar variabilidad en los resultados debido a diferencias en los datos.

Esta variedad de tamaños permite observar el comportamiento de los algoritmos tanto en entradas pequeñas como en entradas grandes. Asimismo, el uso de distintos órdenes iniciales (ascendente, descendente, aleatorio) es fundamental, ya que ciertos algoritmos como QuickSort o SelectionSort pueden presentar rendimientos distintos según la estructura inicial del arreglo y con esto, el caso promedio se vería afectado.

Para la multiplicación de matrices, se generaron matrices cuadradas de tamaños 2^4, 2^6, 2^8 y 2^{10}. Se consideraron tres tipos de matrices: dispersas (principalmente ceros), diagonales (valores sólo en la diagonal principal) y densas (valores aleatorios en todas las posiciones). Cada configuración también cuenta con tres muestras distintas.

Esto permite analizar cómo afecta la estructura interna de los datos al rendimiento de los algoritmos de multiplicación, dado que algunos métodos como Strassen podrían comportarse mejor o peor dependiendo de la cantidad de ceros en las matrices.

3.2. Resultados

4. Resultados experimentales

4.1. Ordenamiento de Arreglos Unidimensionales

La Figura 1 muestra la comparación de tiempo de ejecución entre los cuatro algoritmos de ordenamiento analizados: SelectionSort, MergeSort, QuickSort y std::sort. Los tiempos fueron promediados a partir de tres ejecuciones por caso, y convertidos a segundos. También nos muestra el uso de memoria de estos algoritmos.

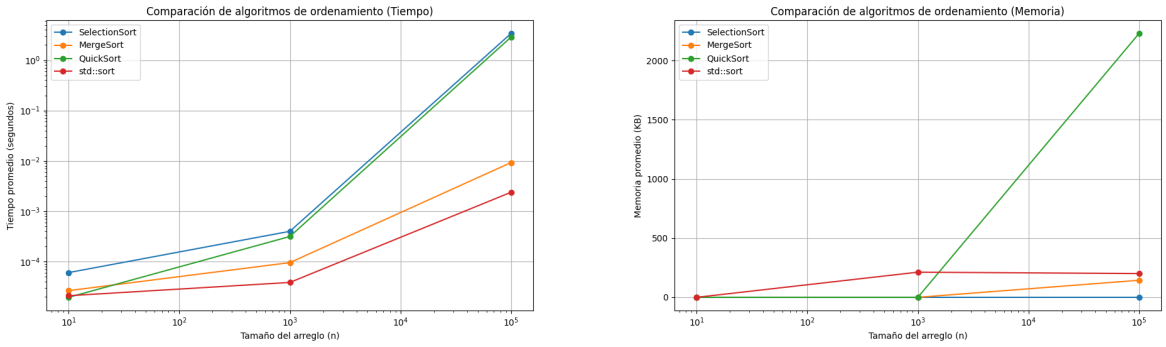


Figura 1: Tiempo de ejecución y uso de memoria de algoritmos de ordenamiento.

En términos de tiempo, `std::sort` demuestra ser el algoritmo más eficiente en todos los tamaños de entrada, seguido por `MergeSort` y `QuickSort`, que presentan un comportamiento muy similar. Por

otro lado, `SelectionSort` muestra un rendimiento significativamente inferior, especialmente a partir de $n = 10^3$, reflejando su complejidad cuadrática. Es importante notar que las curvas se ajustan a los comportamientos teóricos esperados, con los algoritmos $O(n \log n)$ creciendo más lentamente que SelectionSort ($O(n^2)$).

En cuanto al uso de memoria, se observa que `QuickSort` presenta un aumento considerable para $n = 10^5$, lo cual podría deberse a un mal caso de particionamiento o a la implementación que no optimiza la recursión en profundidad. `std::sort` también presenta un uso mayor comparado con SelectionSort y MergeSort, que se mantienen por debajo de los 200 KB en la mayoría de los casos. En particular, `SelectionSort` muestra una eficiencia en memoria coherente con su implementación iterativa in-place.

4.2. Multiplicación de Matrices

La Figura 2 muestra los resultados obtenidos para la multiplicación de matrices.

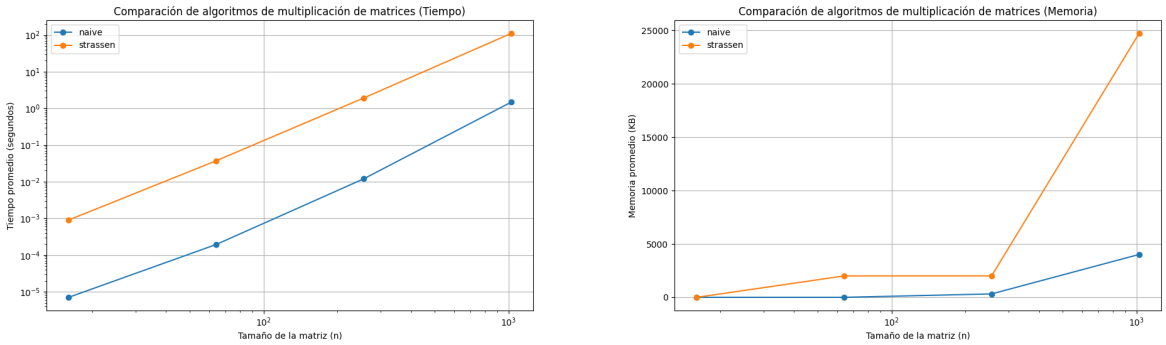


Figura 2: Tiempo de ejecución y uso de memoria en la multiplicación de matrices.

La Figura 2 muestra el tiempo promedio de ejecución de los algoritmos Naive y Strassen en función del tamaño de la matriz. El algoritmo Naive presenta un mejor desempeño temporal, debido a su bajo overhead computacional. Sin embargo, a partir de tamaños medianos y grandes ($n = 2^8$ y $n = 2^{10}$), se observa que Strassen se logra acercar al método Naive en velocidad, gracias a su menor complejidad asintótican sin embargo esto se podria ver mejor con tamaños mas grandes. ($O(n^{\log_2 7}) \approx O(n^{2,81})$ contra $O(n^3)$).

Los resultados muestran que el algoritmo de Strassen tiene un consumo de memoria considerablemente mayor, especialmente para $n = 1024$, donde supera los 24 MB. Esto se explica por la naturaleza recursiva del algoritmo y la creación repetida de submatrices intermedias. En contraste, el método Naive muestra un crecimiento más moderado y estable, reflejando su simplicidad estructural.

5. Conclusiones

Los resultados obtenidos nos permiten evidenciar de forma experimental las diferencias prácticas entre los distintos algoritmos analizados. En el caso del ordenamiento, algoritmos como MergeSort y QuickSort mostraron un rendimiento significativamente superior a SelectionSort, tanto en tiempo como en escalabilidad, mientras que `std::sort`, optimizado a nivel de biblioteca, superó incluso a los algoritmos recursivos clásicos en eficiencia.

El análisis de uso de memoria, por otro lado, mostró que la eficiencia temporal no siempre va de la mano con un bajo uso de recursos. QuickSort, por ejemplo, mostró un pico inesperado en uso de memoria para entradas grandes, lo que resalta la importancia de considerar múltiples métricas al evaluar algoritmos.

Respecto a la multiplicación de matrices, los experimentos mostraron que el algoritmo de Strassen, si bien conceptualmente más eficiente para grandes entradas, incurre en un costo de memoria elevado. El método Naive, en comparación, resultó competitivo e incluso superior en tiempo, mostrando que su simplicidad lo vuelve una alternativa razonable en contextos con pocos datos.

En conjunto, los resultados obtenidos permiten reforzar el entendimiento de que la elección del algoritmo más adecuado depende no sólo de su complejidad teórica, sino también del tipo de entrada, del entorno de ejecución y de los recursos disponibles. Este análisis experimental complementa al estudio teórico y nos entrega una visión más completa sobre el rendimiento de los algoritmos en distintos escenarios.

A. Apéndice 1